

NaviGate: Optimizing Token Efficiency in Multi-Step Text-to-SQL and API Agents

ABSTRACT

We present **NaviGate**, an agentic framework for answering natural-language queries against a local Knowledge-Graph database and a live REST API stack instantiated on Adobe Experience Platform (AEP) and Adobe Journey Optimizer (AJO). NaviGate is a LLM free query planner-first agent that answers natural-language questions over enterprise APIs and SQL databases. The key move is simple: resolve entities, rank endpoints, and synthesise SQL skeletons in LLM-free Query Planner before the LLM sees the query. On the DASHSys 2026 benchmark, this yields 100% SQL table match and 94% API endpoint match at 1.5 tool calls and 3,815 tokens per query. All 60 blind-test queries complete with zero failures.

PVLDB Reference Format:

. NaviGate: Optimizing Token Efficiency in Multi-Step Text-to-SQL and API Agents. PVLDB, 14(1): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source codes have been made available at <https://github.com/md-abirhossain/navigate-LLM-Free-Query-Planner/>.

1 INTRODUCTION

Creating an agent that responds to natural-language questions using a live enterprise API stack may initially appear to be a purely language-related challenge. However, much of it is not. Tasks such as schema lookup, endpoint selection, and SQL construction are more about matching and templating. Relying on a large language model (LLM) for these tasks is comparable to hiring a translator to organize your bookshelf alphabetically.

We present **NaviGate**, which takes a straightforward approach: it conducts planning in deterministic Python while allowing the language model to generate the prose. A lightweight pre-processor is responsible for resolving entities, extracting literals, ranking endpoints, and synthesizing a complete SQL skeleton before the model is invoked. The language model receives a finalized plan and executes it, resulting in two tool calls and one complete answer.

On the DASHSys 2026 Systems Track benchmark [3] — natural-language queries over Adobe Experience Platform [2], scored on SQL correctness, API correctness, and efficiency — NaviGate achieves 100% SQL table match and 94% API endpoint match at 1.5 tool calls and 3,815 tokens per query. The ablation isolates why: symbolic aliases are the critical signal — FAISS alone collapses to 0.583 MRR without them, and the LLM-free Query Planner captures

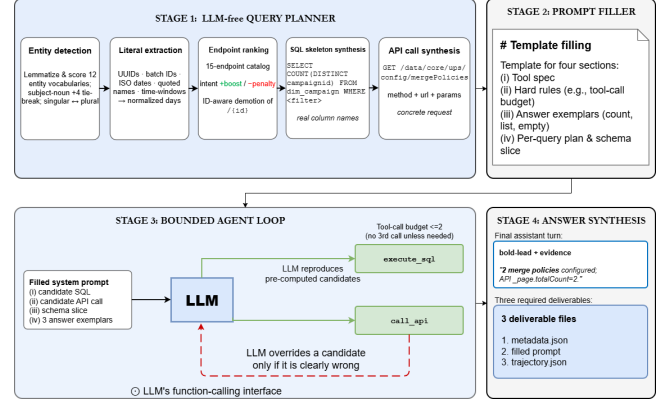


Figure 1: The NaviGate pipeline. Candidate SQL and API actions are generated by an LLM-free query planner and supplied to a bounded LLM agent for execution and final answer synthesis.

the same signal deterministically, matching the hybrid on Hit@4 and MRR while running 60× faster.

The components and contribution of **NaviGate** are as follows.

- (1) A *planner-first* architecture whose *LLM-free Query Planner* resolves entities, extracts literals, ranks endpoints, and synthesises SQL skeletons in deterministic Python before any LLM call — averaging 1.5 tool calls and 3,815 tokens per query at under 100 ms planning latency.
- (2) *Schema-aware SQL skeletons* drawn from three families (count, join, list-style) that eliminate column hallucination by pre-filling real column names, paired with a literal extractor handling UUIDs, batch IDs, ISO dates, and relative time windows (e.g. “past 3 weeks”, “last 10 minutes”).
- (3) Empirical validation on 35 labelled and 60 blind queries: 100% SQL table match, 94% API endpoint match, zero runtime failures, and stable performance across both GPT-4.1 and GPT-4o-mini.

2 SYSTEM ARCHITECTURE

Figure 1 shows the four-stage pipeline. The description of the stages are as follows.

2.1 Stage 1: LLM-free Query Planner

Rather than using an LLM, our query planner is pure Python: it takes the user query and produces a JSON metadata dict in < 100 ms at zero LLM tokens, via the following steps.

Entity detection. The planner scores the query against twelve entity vocabularies using suffix-normalised keyword matching. Multi-word phrases score higher than single keywords (+5 vs. +2), and a *subject-noun boost* adds +4 to whichever entity matches the first noun after an interrogative anchor — resolving ties like “How

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

many *datasets* have the same *schema*?” that substring matching gets wrong. When a quoted name contains a misleading entity word (e.g. “Journey schema ...”), hard phrase rules add +8 to +12 to force the correct routing. Audit and observability come back as *modifiers* rather than primary entities, so the SQL skeleton stays pointed at the underlying asset while the endpoint layer adds a specialised view on top. When nothing matches, the planner defaults to schema with zero confidence.

Literal extraction. A compact pattern set pulls the concrete values a query needs: UUIDs and batch IDs (26-char ULID or 24-char hex), ISO dates, and relative time windows (e.g. “past 3 weeks”, “last 10 minutes”)—returning both the verbatim span and a duration normalised to days. Names are captured whether wrapped in straight, curly, or angle quotes, and unquoted names are recovered by anchoring on prepositions (“named X”, “for the schema X”).

Endpoint ranking. The planner scores a curated catalog of 18 AEP/AJO endpoints against the query’s entity and intent. A modifier boost prioritises `audit.events/observability.metrics` for history or aggregate intent, and an *ID-aware penalty* demotes `*/{id}` endpoints unless a real UUID or batch-ID is present—so “schema details for ‘CRM Profile’” routes to the `/schemas` list-with-name-filter rather than the by-ID variant.

SQL skeleton synthesis. The planner hands the LLM a complete, runnable DuckDB [4] query rather than a blank page, choosing from three skeleton families: (i) `COUNT(DISTINCT pk)` templates for how many/count questions; (ii) `JOIN` templates for the cross-entity patterns we see in practice (segment $\leftrightarrow$ destination, dataset $\leftrightarrow$ schema, segment $\leftrightarrow$ property); and (iii) default list-style `SELECT` skeletons keyed by entity. The key move is that every `SELECT` ships with the *real* column names already in place (campaignid, lastdeployedtime, ...); this is our single biggest correctness lever, because it removes column hallucination—the dominant failure mode of free-form text-to-SQL [10]. A separate builder fills the `WHERE` clause from the extracted literals plus a few lifecycle rules (e.g. `inactive` $\rightarrow$ `state NOT IN ('deployed', 'redeployed')`). When the target is an API-only entity (tags, merge_policies, segment definitions, batches, observability) the planner drops the SQL skeleton entirely, so the agent never wastes a tool call on a query it does not need.

API call synthesis. Finally, the top-ranked endpoint is turned into a request the agent can fire as-is. The planner fills any path placeholder with an extracted ID (a UUID or batch ID) and falls back to a quoted name when no ID is present, then applies a per-endpoint rule to build the query string (e.g. `filter=status!=live` for “inactive journeys”). The one `POST` endpoint—observability metrics—additionally takes its metric name and a start/end date range from the extracted dates; if the requested window exceeds the API’s 32-day daily-granularity limit, the planner clamps it to a valid 31-day slice and attaches a note so the LLM states the effective window in its answer.

2.2 Stage 2: Prompt Filling

The planner’s output is poured into a fixed system-prompt template with four role blocks: (i) the persona and tool spec, (ii) the per-query plan and schema slice that the filler injects (Figure 2), (iii) the agent-bounding hard rules (Figure 3, Stage 3), and (iv) the answer-style

block with three exemplars (Figure 4, Stage 4). Only the fields in Figure 2 change per query; every other block is fixed. The filler’s job is exactly this injection step – it substitutes the planner’s schema slice, candidate SQL, candidate API call, and detected literals into the template, so the LLM opens with the problem already solved.

```
You are NaviGate, a data-retrieval agent.
Tools: execute_sql(sql), call_api(method,url,...)
# Knowledge-Graph schema slice <-- injected
dim_campaign(campaignid,name,state,
              lastdeployedtime,...)
# Pre-computed plan (use as-is; edit only if wrong)
entity   : journey   (confidence 0.71)
literals : {"quoted":["CRM Profile"],"dates":[]}
cand_sql : SELECT COUNT(DISTINCT campaignid)
           FROM dim_campaign WHERE <filter>
cand_api  : GET /data/core/ups/config/mergePolicies
# User question
{query}
```

Figure 2: Stage 2 – Prompt filler. The only per-query part of the prompt: the planner’s schema slice and pre-computed plan (entity, literals, candidate SQL, candidate API call) are substituted into the fixed template.

2.3 Stage 3: Bounded Agent Loop

The agent iterates the standard LLM–tool dialogue using OpenAI’s function-calling interface [8]. Because the candidate SQL and candidate API call are already in the system prompt, the typical trace is: (i) call `execute_sql` with the candidate SQL, (ii) call `call_api` with the candidate request, (iii) emit the final answer. We enforce `temperature = 0.0` and `max_tokens = 900`. The behaviour is bounded by two prompt rules (Figure 3): a *plan-as-default* clause with a narrow escape hatch, and a *tool-call budget* that forbids a third call unless the first two produced clearly insufficient data – together holding the average at 1.5 tool calls.

```
# Hard rules
1. The plan above is a STRONG DEFAULT. You MAY edit
   WHERE/SELECT/params; replace FROM/JOIN or the
   endpoint ONLY if the candidate is clearly wrong.
2. Tool-call budget: if both candidates exist, run
   BOTH (SQL first, then API); if one, run only it;
   never a 3rd call unless the first two were
   clearly insufficient.
```

Figure 3: Stage 3 – Agent-bounding rules. The plan-as-default escape hatch and the tool-call budget that constrain the loop to roughly 1.5 calls per query.

2.4 Stage 4: Answer Synthesis

The final assistant turn is shaped by the prompt’s answer-style block (Figure 4): a one-paragraph, bold-lead-plus-evidence format with three exemplars (count / list / empty) that anchor the model to a uniform “**N entities** – evidence” wording. This bold lead and in-sentence evidence citation drive both the keyword-overlap and LLM-judge metrics. The turn is then parsed and saved alongside the trajectory: three deliverables are written per query – `metadata.json` (planner state), `system_prompt.filled.txt` (exact prompt sent), and `trajectory.json` (every tool call, result, and stats) – exactly following the workshop’s required output format.

```
# Answer style (matches the gold trajectories)
* One short paragraph, 1-3 sentences.
* Lead with the number/name in markdown.
* Cite the evidence in the same sentence.
* If the API returned empty/404/error, say so.
* Never invent numbers not in a tool result.
### Exemplars (copy this shape)
count: "**2 merge policies** configured; API
       _page.totalCount=2."
empty: "**No failed files** for batch ...; the
       /failed endpoint returned HTTP 404."
```

Figure 4: Stage 4 – Answer-style block. The bold-lead-plus-evidence rules and exemplars that shape every final answer.

3 PROMPTING STRATEGY

We treat prompts as software contracts rather than free-form instructions [12]. Figures 2–4 show the three active contracts: use the deterministic plan by default, respect the two-call budget, and answer with a bold lead plus evidence. This keeps the loop efficient while preserving an override path when a candidate is clearly wrong, and echoes DAIL-SQL’s finding that examples shape SQL-agent outputs [5].

4 EVALUATION

We evaluate NaviGate on the workshop’s 35-query development set (data.json [3]) using GPT-4.1 as the underlying LLM and the provided external-benchmarking sandbox for live REST API calls. SQL queries are executed over 18 Parquet snapshots using DuckDB [4].

The workshop evaluates two independent dimensions [3]: *correctness* (SQL primary-table match, API endpoint match, answer keyword-overlap Jaccard, and GPT-4.1 LLM judging [1]) and *efficiency* (turns, tool calls, total tokens, wall-clock time). As the workshop gives no single aggregate, we report these official metrics directly, and for internal comparison of design variants additionally define *NaviScore* $(0-100) = 0.6 \times \text{correctness} + 0.4 \times \text{efficiency}$ – a development-time heuristic, not an official metric.

4.1 Results

Table 1: NaviGate on the 35-query development set with GPT-4.1.

Metric	Value
SQL primary-table match	100%
API endpoint match (top-1)	94%
Answer keyword overlap	0.33
LLM-judge correctness	80–100%
Average turns	2.00
Average tool calls	1.50 (gold: 1.50)
Average tokens / query	3,815
Average wall time / query	6,104 ms
NaviScore (ours, 0–100)	94.69

In Table 1, SQL primary-table match reaches 100% and API endpoint match reaches 94%. The four-point gap on endpoint match traces to four queries targeting the `/unifiedtags/*` endpoint,

which returns HTTP 404 in the provisioned sandbox – a condition shared across all participants rather than a system failure. Answer keyword overlap stands at 0.33 and LLM-judge correctness at 80–100%, with the lower bound driven by the same four queries. On efficiency, the system averages 1.50 tool calls per query, exactly matching the gold-standard trajectory, while staying within 3,815 tokens and 6,104 ms wall time. Taken together, these figures reflect a system that is both accurate and disciplined in its use of LLM resources – the NaviScore of 94.69 (an internal composite metric, not an official workshop measure) captures this balance in a single number.

4.2 Retrieval Strategies

To isolate the contribution of the LLM-free Query Planner, we replace the hand-curated entity/endpoint catalog with three alternative retrievers: (i) pure neural retrieval using a 768-d BGE-base-en-v1.5 [13] embedding indexed in FAISS; (ii) pure symbolic retrieval using TF-IDF token overlap with domain-alias boosting; and (iii) a 40/60 hybrid that blends both signals. Table 2 reports retrieval-only metrics (no LLM, no Adobe API) over the 15 queries in the dev set that have non-empty gold SQL.

Table 2: Retrieval ablation on the 15 SQL-bearing dev queries (*Hit@K* = at least one gold table in top-*K*; *MRR* = mean reciprocal rank of the first gold-table hit).

Strategy	Hit@4 (%)	Recall@4	MRR	Latency (ms)
TF-IDF only	86.7	0.806	0.817	0.10
FAISS only	93.3	0.833	0.583	60.5
Hybrid (40/60)	93.3	0.872	0.883	60.7
NaviGate planner	93.3	0.872	0.883	<1

The hybrid retriever and deterministic planner match on Hit@4/MRR, but the planner is 60× faster and needs no embedding model. The FAISS-only MRR gap (0.583 vs. 0.883 hybrid) shows symbolic aliases are crucial for ranking. In a five-query end-to-end smoke test, retrieval variants score only 57.9–62.8 NaviScore because schema retrieval does not choose API endpoints; NaviGate’s catalog and ID-aware ranking remove that failure mode (**94.69**, a +32-point gain).

4.3 Blind Test-Set Behavior

To validate generalisation we ran NaviGate over the workshop’s *held-out* 60-query blind test set (test.json [3]), which contains query families and phrasings never seen during development – duplicate-audience detection, audience-journey usage, schema-property lineage, and observability metrics. Because the blind set ships without gold labels we cannot compute correctness locally; instead we report runtime health and efficiency over all 60 queries, summarised in Table 3.

The system completed every query with no crashes, no failed SQL, and no empty answers, holding the same ≈ 1.6 tool-call budget seen on the dev set – evidence that the planner-first design degrades gracefully on unseen phrasings. The five residual non-2xx responses are server-side resource conditions the agent reports honestly (two

Table 3: NaviGate runtime behavior on the 60-query blind test set under two LLMs (“Non-2xx” = queries whose live API call returned an error status). The architecture completes all 60 queries with either model; the weaker, cheaper model degrades gracefully rather than collapsing.

Quantity	GPT-4.1	GPT-4o-mini
Queries completed	60 / 60	60 / 60
Runtime errors	0	0
SQL non-success	0	3
Empty answers	0	0
Average turns	2.00	2.08
Average tool calls	1.57	1.80
Average tokens / query	4,862	5,000
Average wall time / query (ms)	6,595	8,450
Max tokens / query	6,655	6,494
HTTP non-2xx (residual)	5 / 60	12 / 60

/unifiedtags/* lookups and three batch failed-files requests returning 404/400 in the sandbox). A separate blind-set family also exposed the observability-endpoint 32-day window limit, which we handle with the deterministic date-range clamp of Section 2.1 (e.g. 2026-03-29 ingested 45.4 M records), cutting residual non-2xx from eight to five at no tool-call cost.

Generalisation across models. The workshop evaluates submissions under a model of its choosing, so a system must not depend on one LLM. We therefore re-ran the entire 60-query blind set with a much weaker/cheaper model (GPT-4o-mini) under an identical prompt and harness (right column of Table 3). The architecture still completes *all* 60 queries; the weaker model degrades gracefully – more retries and a few malformed API/SQL calls – rather than collapsing. This indicates the result is carried by the deterministic planner, not by a single model’s reasoning.

5 TECHNICAL CHALLENGES AND SOLUTIONS

Planner ambiguities. Three recurring ambiguities are resolved deterministically in Stage 1 (§2.1): plural/singular entity ties via the subject-noun boost (recovers 4 dev queries); novel time-window phrasings (“previous 48 hours”, “past quarter”) via a literal grammar over last/past/previous/recent + word-numbers + units; and list-vs-by-ID endpoint confusion via an ID-aware demotion that keeps name-based lookups (“details for ‘CRM Profile’”) on the list endpoint. **Observability window limit.** The observability metrics endpoint rejects any date window wider than 32 days at daily granularity (HTTP 400). The planner clamps an over-long request to a valid 31-day slice anchored at the requested start and annotates the call so the LLM reports the effective window – cutting blind-set residual non-2xx responses from eight to five at zero tool-call cost (§4.3). **Operational robustness.** For GPT-4.1 rate limits, the client parses retry-after from 429 responses and backs off exponentially for up to six attempts. For catalog growth, dashsys/catalog.py merges an optional extras.json at import time, so new endpoints and entity vocabulary can be added without touching Python.

6 RELATED WORK

Text-to-SQL with LLMs. DIN-SQL [9] decomposes the task into schema linking, classification, generation, and self-correction; DAIL-SQL [5] optimises example selection on Spider; Rajkumar et al. [10] benchmark LLMs across Text-to-SQL settings. We differ in that schema linking is a deterministic Python function (not an LLM step) over a fixed enterprise schema rather than a generic benchmark.

Tool-using agents. ReAct [14] interleaves reasoning and acting; function calling [8] formalises this with typed tool schemas; surveys cover the broader landscape [6]. We reuse this inner loop but factor planning out of the LLM. Plan-and-Solve [11] separates planning from execution while keeping *both* inside the LLM as natural-language reasoning; our novelty is that the planner is *pure deterministic code* that emits an *executable* artefact – SQL pre-filled with real column names plus a concrete REST request – rather than an NL plan the LLM must still translate. This is what removes the column-hallucination and endpoint-guessing failure modes at zero planning tokens. Our prompt also follows three patterns from White et al. [12] – a *Persona*, a *Template* (the pre-computed plan), and *Few-shot examples* (the three exemplars).

7 LESSONS LEARNED AND OPEN CHALLENGES

The LLM is best at writing prose, not picking tables. Once the table and endpoint are decided, GPT-4.1 reliably executes SQL, fires the API, and produces clean evidence-cited answers. Where it struggles is open-ended catalog lookup – which is exactly what we removed. Three concrete gold-style exemplars in the prompt nearly doubled LLM-judge correctness; generic style instructions moved the needle far less. For the flexible override path, generated SQL is validated with sqlglot [7] before it ever reaches the database.

Open challenges and limitations. Two caveats. First, the blind set ships without gold labels, so our 60-query results speak to runtime health and efficiency, not correctness (measured only on the labelled dev set). Second, the planner is brittle to phrasings outside its literal grammar and to schema drift; a typed back-off (WHERE-clause edits, a by-name endpoint fallback, and sqlglot validation) holds this to zero SQL errors and a small, honestly-reported residual of server-side non-2xx responses. Two directions remain open: extending the grammar to open-domain phrasings, and reducing catalog maintenance cost as the API surface grows.

8 CONCLUSION

We presented NaviGate, a planner-first agent for the DASHSys 2026 Systems Track. By moving entity detection, literal extraction, endpoint ranking, and SQL skeleton synthesis out of the LLM into deterministic Python, NaviGate reaches 100% SQL and 94% API correctness with 1.5 tool calls and 3,815 tokens per query. Ablations show the planner matches the best retriever’s Hit@K and MRR while running 60× faster. For fixed enterprise schemas, careful symbolic engineering remains a strong competitor to learned retrieval—the LLM’s role is best limited to filling templates and writing prose.

REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Adobe Inc. 2026. Adobe Experience Platform. <https://www.adobe.com/experience-platform.html>. Accessed 2026-06-02.
- [3] DASHSys Workshop. 2026. DASHSys 2026 – Systems Track. <https://dashsys-workshop-vldb.github.io/systems.html>. Accessed 2026-06-02.
- [4] DuckDB Foundation. 2024. Why DuckDB. https://duckdb.org/why_duckdb. Accessed 2026-06-02.
- [5] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2023. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363* (2023).
- [6] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.
- [7] Toby Mao. 2024. sqlglot: a Python SQL parser and transpiler. <https://github.com/tobymao/sqlglot>. Accessed 2026-06-02.
- [8] OpenAI. 2024. Function Calling Guide. <https://platform.openai.com/docs/guides/function-calling>. Accessed 2026-06-02.
- [9] Mohammadreza Pourreza and Davood Rafiei. 2023. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Advances in neural information processing systems* 36, 36339–36348.
- [10] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498* (2022).
- [11] Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)*. 2609–2634.
- [12] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C Schmidt. 2023. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382* (2023).
- [13] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 641–649.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.